tf.compat.v1.profiler.Profiler Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/profiler/Profiler

> *TensorFlow multi-step profiler.*

Function Signatures

```
tf.compat.v1.profiler.Profiler( graph=None, op_log=None )
```

Code Examples

```
tf.compat.v1.profiler.Profiler(
graph=None, op_log=None
)
```


```
tf.compat.v1.profiler.Profiler(
graph=None, op_log=None
)
```

```
Typical use case:
# Currently we are only allowed to create 1 profiler per process.
profiler = Profiler(sess.graph)
for i in range(total_steps):
if i % 10000 == 0:
run_meta = tf.compat.v1.RunMetadata()
_ = sess.run(...,
options=tf.compat.v1.RunOptions(
trace_level=tf.RunOptions.FULL_TRACE),
run_metadata=run_meta)
profiler.add_step(i, run_meta)
# Profile the parameters of your model.
profiler.profile_name_scope(options=
(option_builder.ProfileOptionBuilder
.trainable_variables_parameter()))
# Or profile the timing of your model operations.
opts = option_builder.ProfileOptionBuilder.time_and_memory()
profiler.profile_operations(options=opts)
# Or you can generate a timeline:
opts = (option_builder.ProfileOptionBuilder(
option_builder.ProfileOptionBuilder.time_and_memory())
.with_step(i)
.with_timeline_output(filename).build())
profiler.profile_graph(options=opts)
else:
_ = sess.run(...)
# Auto detect problems and generate advice.
profiler.advise()
```

```
Typical use case:
# Currently we are only allowed to create 1 profiler per process.
profiler = Profiler(sess.graph)
for i in range(total_steps):
if i % 10000 == 0:
run_meta = tf.compat.v1.RunMetadata()
_ = sess.run(...,
options=tf.compat.v1.RunOptions(
trace_level=tf.RunOptions.FULL_TRACE),
run_metadata=run_meta)
profiler.add_step(i, run_meta)
# Profile the parameters of your model.
profiler.profile_name_scope(options=
(option_builder.ProfileOptionBuilder
.trainable_variables_parameter()))
# Or profile the timing of your model operations.
opts = option_builder.ProfileOptionBuilder.time_and_memory()
profiler.profile_operations(options=opts)
# Or you can generate a timeline:
opts = (option_builder.ProfileOptionBuilder(
option_builder.ProfileOptionBuilder.time_and_memory())
.with_step(i)
.with_timeline_output(filename).build())
profiler.profile_graph(options=opts)
else:
_ = sess.run(...)
# Auto detect problems and generate advice.
profiler.advise()
```

```
add_step(
step, run_meta
)
```


```
add_step(
step, run_meta
)
```

```
advise(
options
)
```


```
advise(
options
)
```


Documentation

TensorFlow multi-step profiler.

Methods

add_step

View source

Add statistics of a step.

advise

View source

Automatically detect problems and generate reports.

profile_graph

View source

Profile the statistics of graph nodes, organized by dataflow graph.

profile_name_scope

View source

Profile the statistics of graph nodes, organized by name scope.

profile_operations

View source

Profile the statistics of the Operation types (e.g.

MatMul, Conv2D).

profile_python

View source

Profile the statistics of the Python codes.

By default, it shows the call stack from root. To avoid
redundant output, you may use options to filter as below
options['show_name_regexes'] = ['.my_code.py.']

serialize_to_string

View source

Serialize the ProfileProto to a binary string.

Users can write it to file for offline analysis by tfprof commandline
or graphical interface.